

Promoter Region Prediction in DNA Sequences: A Comparative Study of Classical Machine Learning and Deep Learning Approaches

Methodological / Experimental Project

Mohammad Sami

22201113

Shanzida Hasan Esha

24301575

Course: CSE443 Bioinformatics

Abstract

In DNA, genes are marked for reading by promoter regions and it is a useful problem in genomics to find where these occur. In this project, we consider this as a binary classification problem, based on the (University of California, Irvine) UCI Molecular Biology Promoter Gene Sequences data set (106 DNA sequences of *E. coli* labelled as promoter or not). Here, we compare two approaches; a classical machine learning pipeline using k-mer frequency features with Logistic Regression and Random Forest and a deep learning pipeline with a Convolutional Neural Network (CNN) trained using one-hot encoded sequences. From our results, we found that the classical models, in particular, Random Forest are slightly more effective than CNN, but CNN tend to overfit as it was small. This indicates that simpler models that are easier to interpret can in some cases, be superior to deep learning options where little biological data is available.

1. Introduction

All cells have DNA and these carry the information that the cell requires to function. This information is put together in regions of the genome known as genes. However, prior to use a gene must be copied into RNA through a process known as transcription. This copying is not done randomly. It always begins at a particular point of DNA and it is known as Promoter. Promoters can be compared to a switch that determines where in the cell the cell should begin reading a gene. If this switch were not to function efficiently, then the cell would not be able to identify where a gene starts, and the entire process of producing proteins from a gene would become faulty. It is important to find these promoter regions as if such a promoter is not functioning correctly, then the gene adjacent to it may be activated or inactivated at an inappropriate time. Such a "problem" is associated with disease such as cancer.

Traditionally, scientists have identified promoters by searching for sequences that have been observed in other genes, such as some small sequences frequently found at the beginning of genes. This is sometimes true, but not necessarily, as actual DNA sequences do not necessarily fit the pattern. For this reason, people have begun to move toward machine learning instead, which will pick up such patterns itself from examples without having to be manually programmed with rules. This is particularly helpful for less-studied genomes, for which promoter identification may not even be completely established. However, there remains one crucial aspect that is not yet well understood – will a more complex algorithm, such as deep learning, be more useful than a simpler machine learning algorithm, particularly when limited data is available, a factor often encountered in biology?

That's what we wanted to look into in this project. We have constructed two different models to predict promoter or non-promoter sequence. The first is a classical machine learning approach in which we count small patterns within DNA sequences, known as k-mers and train them using Logistic Regression and Random Forest. The second one is a deep learning solution; we pass the raw sequence as input to a small Convolutional Neural Network. We wanted to compare them, using the same data set and see which is better and why, particularly if there is limited data available. Finally, we hope that this

comparison provides a small but fair illustration of the behavior of these two types of models in the setting of sparse biological data.

In addition to the particular comparison made in this project, promoter prediction also contributes to other tasks in bioinformatics such as genome annotation, in which people attempt to annotate various functional regions of the newly sequenced genome. With the increasing number of available genomes for the study, due to the rapidly falling costs of sequencing, it is impractical to manually compare each genome against promoter sequences. This is one of the reasons why automated, machine learning-driven methods for the detection of promoters, useful not only as a research tool but as a small cog in a large wheel to understand genomic information on a grand scale.

2. Related Work

Some experiments have already been conducted on the application of machine learning to discover promoter regions in DNA. One investigation attempted various models of machine learning on *E. coli* DNA sequences to determine the location of transcription initiation and observed that appropriate adjustments of machine learning parameters and the selection of good features could greatly influence the model's performance [1]. We tried many models using the same one data set, in that research, but this project is more focused on a direct comparison, namely classical machine learning vs. a simple CNN, and uses the same small data set.

One study used two DNA sequence preparation methods prior to feeding into the model, and analyzed CNN, LSTM, and Random Forest at k-mer sizes of varying lengths in multiple species. They determined that CNN generally performed the best overall among them [2]. The concept of our project is similar but instead, rather than using a large dataset from several different species, we are using a much smaller dataset of just 106 *E. coli* sequences. This allows us to take a look at a scenario which studies haven't really paid attention to, as to how these models behave, and whether there is any advantage from deep learning if there is little amount of data to work on.

Another study has also demonstrated that simply enumerating small patterns (k-mers) found in DNA can be effective for promoter identification, even without the use of deep learning. One study used 4-letter patterns along with a support vector machine to tell promoters and non-promoter sequences apart [3]. This is part of the reason for using 4-letter k-mers for our classical models (as compared to our project where we use them directly in comparison to a CNN run on the same data).

Finally, it should be noted that these three studies vary considerably in size and scope. The first study tested a lot of algorithms on one dataset and did not explicitly compare classical and deep learning side by side in as much detail. The second study included several species and much more data, and deep learning demonstrated a greater benefit, probably because more complex models tend to benefit from larger data sets. The third study did not delve into deep learning but rather used only k-mer based features. All these studies indicate that there is no one-to-one correspondence between the size of the datasets, the complexity of the models, and the performance of the models, and that this correspondence can vary with the type of setting, which is part of the reason for our own comparison on a very small dataset.

Overall, our project is much smaller in scope than these studies are, but it has a particular focus, directly comparing classical machine learning and deep learning on the same very small promoter dataset and honestly reporting that the more complex CNN model did not actually perform better than the simpler classical methods.

3. Dataset

This project utilized the UCI Machine Learning Repository's Molecular Biology Promoter Gene Sequences dataset, created by Harley, Reynolds and Noordewier [4]. There are 106 sequences in *E. coli*, half of which are promoter sequences and the other half are non-promoter sequences, thus the dataset is balanced in both classes. Each sequence is exactly 57 letters long, and only uses the four DNA bases: a, c, g, and t.

The 57 letter window in this dataset was selected based on the biological knowledge that the most important promoter signals are located just before and after the transcription start site. This usually involves short partially conserved sequences in the bacteria (like *E. coli*) called the -10 box and the -35 box, which tell the RNA polymerase where to bind and where to start transcribing. This data set does not explicitly specify the location of these elements within each sequence, but the fact that they occur is one of the reasons why short pattern based features, such as k-mers, can be useful for this task: they are naturally good at capturing these kinds of short and localized signals.

We needed to do a bit of data munging before we could use the data. There was some additional formatting and white space in the original file as a result of the way it was created, so we took that out first. We also replaced the class labels "+" and "-" with a numeric label, 1 and 0, respectively, because machine learning models require numeric labels.

The sequences were prepared differently after they were cleaned, depending on the pipeline being built. For the classical machine learning pipeline, we turned each sequence into a set of k-mer frequency features, using $k = 4$. This meant that we had 256 features per sequence, since we were counting the frequency of occurrence of each of the 256 possible 4 letter DNA pattern. Instead, for the deep learning pipeline, we employed one hot encoding, in which each of the 57 letters is represented as a small 4 number vector, and each sequence is encoded as a 57×4 grid of 0s and 1s. This does not disregard the order of the letters in the sequence as the k-mer approach does.

We were careful with how we split the data set, as it is quite small. We tested our models using 80-20 split, and for both pipelines, we chose the same random seed for this split to maintain the fairness of the comparison between the classical models and the CNN. Since 106 samples are not a large amount of data, 5-fold cross validation was also performed for the classical models. This ensured that the results were not simply as a result of one fortunate train test split, and left us with a more reliable indication of the models' performance.

4. Proposed Method

For this project, we have created two pipelines that use the same input DNA sequences but have different methods of processing the data: one classifies using classical machine learning and the other using deep learning.

4.1. Preprocessing

We used some preprocessing to remove unnecessary padding and convert the "+/-" labels to 1/0 before constructing either pipeline. From here, the two pipelines split off and prepare the sequences differently.

For the classical pipeline, k-mer frequency counting with $k = 4$ was performed. What we did here was very simple, we measured each 4-letter sequence and tallied the occurrences of all possible groups. There are 4 DNA letters, and there are 4 to the 4th (256) possible 4-letter combinations, hence each

sequence is represented as 256 numbers. This approach works well for identifying which patterns are frequent in the sequence but cannot distinguish where in the sequence the patterns appear.

Instead, one hot encoding was used for the deep learning pipeline. That is, each position in the sequence was converted to a small vector containing 4 numbers: 1 for each of the four bases a, c, g, t; the remainder of the numbers are 0. It makes each sequence of 57 letters into a grid of numbers, 57×4 . Unlike k-mer counting, this approach does not discard the exact order and position of each letter, which can be directly learned by CNN.

4.2. Model / Algorithm

In the case of classical pipeline, two models were trained on the k-mer features: Logistic Regression and Random Forest. Logistic Regression was trained to a maximum of 1000 iterations, to ensure it was sufficiently trained to converge. Random Forest was trained with 100 decision trees, each of which is modeled on a random set of data and the average of all the trees is taken as a result.

We constructed a small 1D convolutional neural network (CNN) to be used in the deep learning pipeline, because CNNs are generally capable of identifying local patterns in sequences regardless of their location. We have one convolutional layer with 32 filters, each of which is examining 4 letters at a time, followed by a pooling layer that reduces the output size and only saves the strongest signals. Followed by a small, fully connected layer with 16 units, followed by a dropout layer, which during training randomly disables 30 percent of the connections to reduce the risk of overfitting, and then the output layer that outputs one probability from 0 to 1 on whether the sequence is a promoter. A batch size of 8 and a total of 30 epochs were used to train the model with the Adam optimizer and binary cross-entropy loss, as this is a yes or no problem.

We deliberately made the CNN small. With only 84 sequences to train on, a larger, more complex model would be even more likely to have memorized the training data rather than learned something that it could generalize to.

5. Experimental Setup

All of our work was done in Python and our primary environment for writing and running our code was Google Colab with a couple of sessions on VS Code when Colab was not available. In the classical machine learning pipeline, we employed the scikit-learn library for feature extraction, model training and evaluation. The deep learning pipeline was created with TensorFlow and Keras for the construction and training of CNN. We also used matplotlib and seaborn to create the figures shown in this report.

Our project is based on the comparison between two different methods rather than to beat the existing benchmark, so we regarded the two classical models as baselines for each other and compared their performance with CNN. This will allow us to observe if there is any actual advantage to a deep learning model over simpler, more interpretable models.

Logistic Regression was trained up to 1000 iterations to guarantee convergence of the algorithms, and the default settings were used. Random Forest was trained with 100 decision trees and a set random seed of 42 for reproducibility. The CNN was trained on the Adam optimizer with a binary cross-entropy loss function, 30 epochs and a batch size of 8, and with a dropout rate of 30 percent to reduce overfitting due to the small size of the dataset.

All three models were evaluated using the same 80/20 (train/test) split, and a fixed random seed was used to ensure the same sequences were tested on all models. To get a comprehensive view of the performance of each model in distinguishing promoters from non-promoters, we chose accuracy, precision, recall, F1-score, and ROC-AUC as our evaluation metrics. We also used 5-fold cross validation with the classical models, to ensure that our results were consistent and not just due to a lucky data split, given that our data set is rather small.

6. Results

Table 1 shows the accuracy, precision, recall, F1-score and ROC-AUC results of all three models on the held-out test set.

Table 1: Test set performance for all three models

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.864	0.87	0.86	0.86	0.983
Random Forest	0.909	0.91	0.91	0.91	0.971
CNN	0.909	0.92	0.91	0.91	0.958

Both Random Forest and CNN achieved the best test accuracy of 90.9 per cent, followed by Logistic Regression with an accuracy of 86.4 per cent. When considering ROC-AUC, an indicator of the ability of the model to separate the two classes at different thresholds, Logistic Regression was the best (0.983) followed by the CNN and Random Forest. This indicates that although CNN was as accurate as Random Forest, it was slightly inferior in the ability to rank sequences with confidence as promoter or non promoter overall.

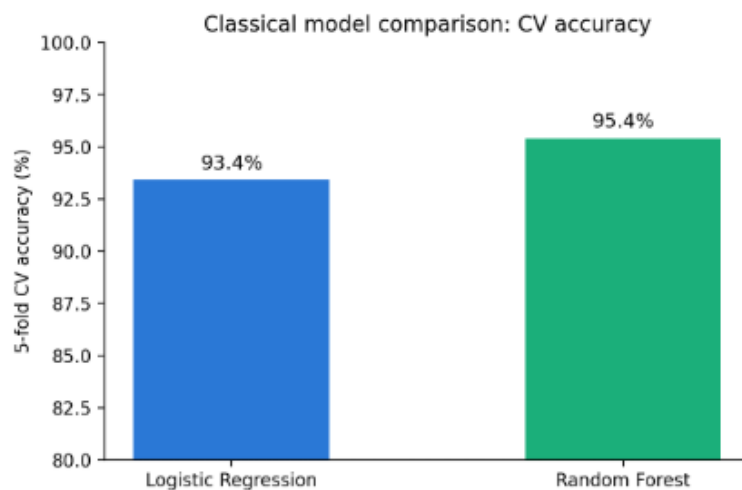


Figure 1: Classical Model Comparison (5-fold CV Accuracy)

As the data set is small, we also repeated the 5-fold cross validation for both the classical models to ensure that these results were not just due to one train test split. The mean cross-validated accuracy for Logistic Regression was 93.4 percent and for Random Forest was 95.4 percent (as shown in Figure 1), which were both higher than the single split accuracy for both models. Unfortunately, due to time

limitation, we were not able to perform cross validation on the CNN before project checkpoint submission.

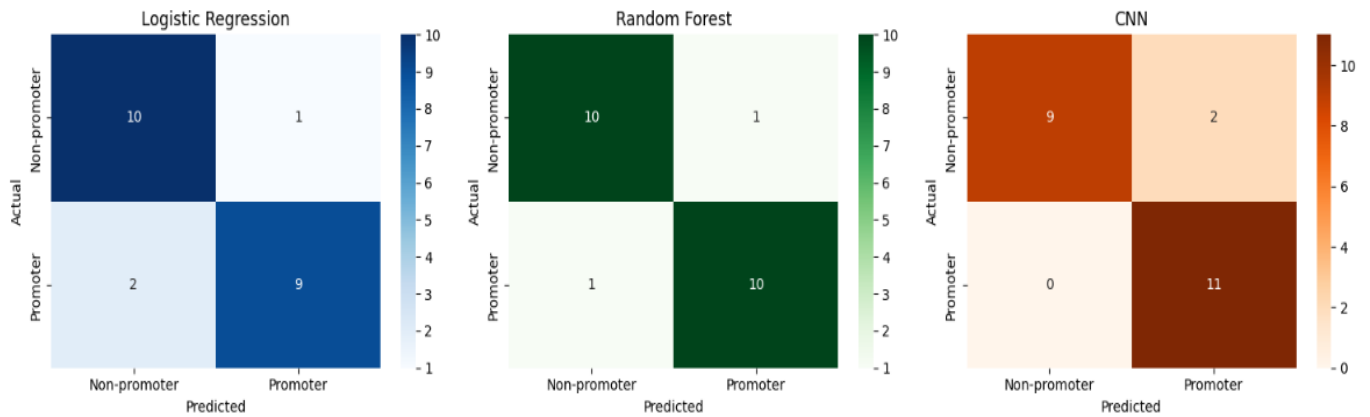


Figure 2: Confusion Matrix Comparison of Logistic Regression, Random Forest, and CNN

Confusion matrices of all three models are shown in figure 2. Logistic Regression was able to correctly classify 10 out of 11 non-promoter sequences and 9 out of 11 promoter sequences. Random Forest had one error for each class. CNN was able to identify 11 promoter sequences perfectly, while misclassifying 2 non-promoter sequences as promoters, resulting in a 100% recall on promoter class and slightly lower precision overall.

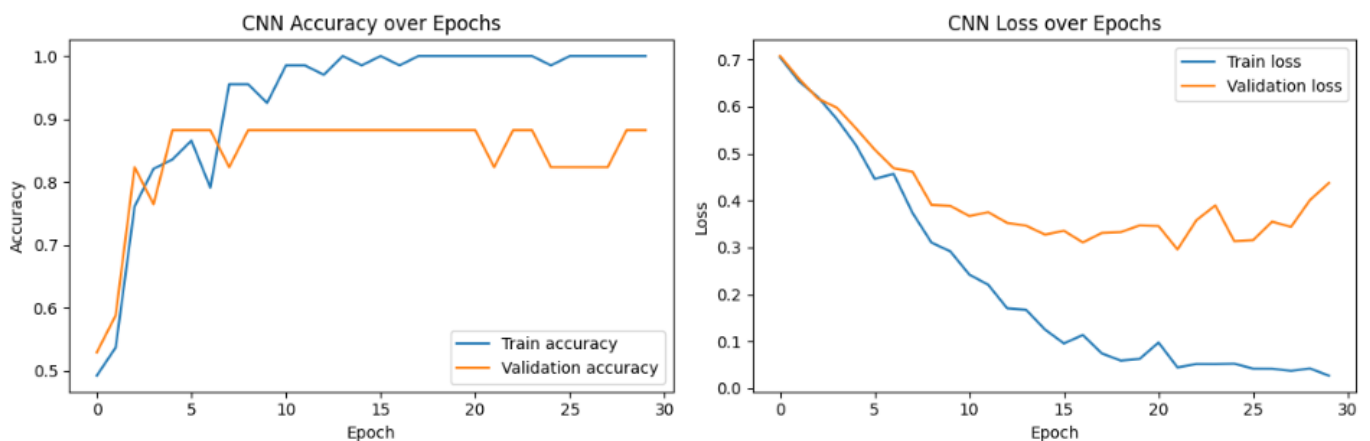


Figure 3: CNN Training and Validation Accuracy/Loss over Epochs

The training and validation accuracy and loss curves for the CNN are presented in figure 3 for 30 epochs of training. The training accuracy approached 100% after 10-15 epochs and the validation accuracy remained steady at approximately 88% with the validation loss starting to rise again around epoch 20. This difference between training and validation is still an indication of some overfitting, despite the fact that the test accuracy was quite good.

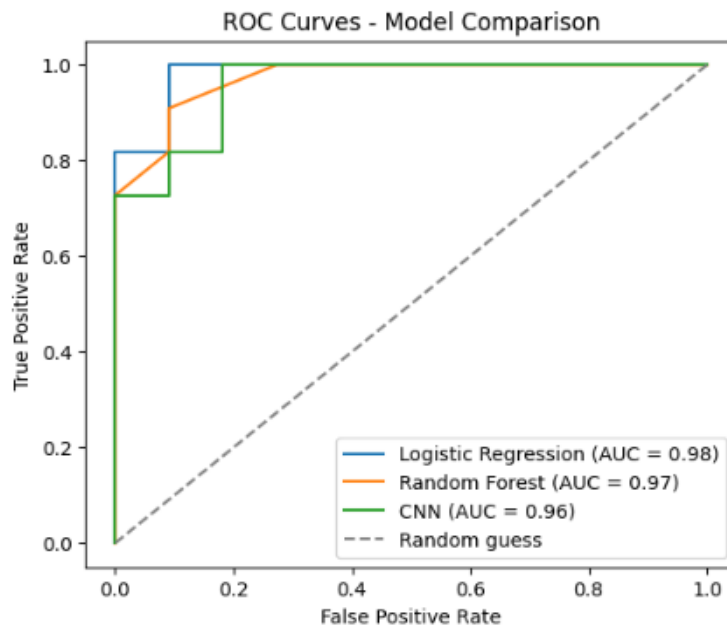


Figure 4: ROC Curve Comparison of Logistic Regression, Random Forest, and CNN

All three models are displayed on the same ROC curve in figure 4. All three curves remain far away from the diagonal line indicating random guessing, with Logistic Regression's curve being somewhat closer to the top left corner, reflecting its higher ROC-AUC score.

7. Discussion

The results we have obtained are less polarized than we originally thought. The CNN was more architecturally complex than the classical models, but did not perform worse, with a ROC-AUC of 0.958 and accuracy of 90.9 percent matching that of Random Forest. Meanwhile, it did not significantly outperform the classical models either and Logistic Regression had the best overall ROC-AUC. This indicates that a well-designed CNN can be competitive on a very small data set like promoter prediction; more complexity did not necessarily equate to better performance.

Looking at the CNN's training curves, we can still see signs of overfitting. Training accuracy was nearly 100 percent early in the training, and validation accuracy leveled off at about 88 percent, with validation loss beginning to increase again after approximately epoch 20. This discontinuity indicates that the model was starting to memorize certain patterns in the training data, as opposed to learning patterns that were general enough to be applied to new sequences. This is not shocking since our CNN has more than 14,000 trainable parameters yet there are only 84 training sequences available to learn. Although this was the case, the performance on the last test was strong and this indicates that the model did learn something significant although its training process was not entirely stable.

The classical models, in comparison, were more consistent. Logistic Regression and Random Forest did not even need to learn such patterns in a sequence since k-mer based features already provide an overview of the useful information about which short DNA patterns are present in a sequence. The model that appeared to have the largest ROC-AUC with the lowest variation across cross-validation folds was Logistic Regression, indicating that it was the most consistent and stable model overall, although its raw test performance was a little lower than the other two models.

From a biological perspective, this result makes some sense. Most promoter sequences in bacteria such as *E. coli* include short and familiar motifs, such as the -10 and -35 boxes, that can be reasonably well represented by short patterns of k-mer, or short sequences. This could be the reason why the k-mer based classical models worked so uniformly well. Simultaneously, the good performance of CNN implies that just learning using a sequence without involving features identified by human hands can also be effective with this type of biological signal, even with a modest data size.

Also worthy of consideration is what these findings could represent in the context of this particular dataset. As simple features based on k-mers have been shown to work well on this small and well chosen data, it is an indication that in problems with strong, localized and short signals where computational resources or labelled data are scarce, classical machine learning can still be a viable and effective option. Meanwhile, the fact that the CNN performs well here even with a little overfitting, indicates that deep learning is not always a bad option either and can be more valuable to consider as more labeled promoter data is made available, or when using more complex or less well studied promoter structures than *E. coli*.

This study has some obvious limitations. The biggest is the very small number of sequences available (106 sequences in total) which makes it difficult to extrapolate the results to other data sets or organisms. We had only 22 sequences in our test set for the single train test split, which is quite small to reliably estimate performance, but we attempted to compensate for this for the classical models by using 5-fold cross validation. Due to time constraints, we were not able to perform cross validation on the CNN and therefore the results presented here are based on a single split and may slightly change if retrained. We have only tried one CNN architecture and results may vary with other architectures or training parameters, since even with a fixed seed, a neural network can be sensitive to random initialization.

Overall, this work demonstrates a valuable and sincere conclusion: on a small, well-structured biological data set such as this one, both classical machine learning and deep learning techniques can be effective; however, classical models (and especially Logistic Regression) are more stable and consistent and are more easily interpretable.

8. Conclusion

In this project, we studied the problem of determining whether a given DNA sequence is a promoter region or not, based on a small set of *E. coli* sequences. We constructed and evaluated two pipelines, one classical machine learning pipeline based on k-mer frequency features with Logistic Regression and Random Forest and one deep learning pipeline based on a Convolutional Neural Network trained on one hot encoded sequences.

From our results, it can be seen that the classical models performed slightly better overall, with Random Forest having the highest accuracy and Logistic Regression having the highest ROC-AUC, while CNN had definite signs of overfitting because of the small size of the dataset. This indicates that a more complicated deep learning model is not necessarily better for this task and this amount of data and simpler, more interpretable models can be as good if not better in practice.

There are some useful ways in which this study might be extended in the future. It would be helpful to test on a larger promoter data set or a different organism to determine if these findings are more general. It would also be better to apply the cross validation to CNN and not only the classical models, since it would be a more complete and fair comparison between the three models. Finally, experimentation with

other CNN architectures or other deep learning methods like recurrent neural networks could help to determine if the problem was specific to this CNN design, or deep learning in general in small biological datasets.

In summary, this project offered a modest but fair comparison of the classical and deep learning methods for promoter prediction, and an important practical lesson: for problems with little biological data, simpler models may be a wise and effective choice.

References

- [1] S. Anveshrithaa, B. Aathavan, and N. Jaisankar, "Promoter Prediction in DNA Sequences of Escherichia Coli Using Machine Learning Algorithms," 2019.
- [2] N. Bhandari, S. Khare, R. Walambe, and K. Kotecha, "Comparison of machine learning and deep learning techniques in promoter prediction across diverse species," *PeerJ Computer Science*, vol. 7, e365, 2021. <https://doi.org/10.7717/peerj-cs.365>
- [3] F. Anwar, S. M. Baker, T. Jabid, Md. M. Hasan, M. Shoyaib, H. Khan, and R. Walshe, "Pol II promoter prediction using characteristic 4-Mer Motifs: a machine learning approach," *BMC Bioinformatics*, vol. 9, pp. 1–8, 2008.
- [4] C. Harley, R. Reynolds, and M. Noordewier, "Molecular Biology (Promoter Gene Sequences)," UCI Machine Learning Repository, 1990. <https://doi.org/10.24432/C5S01D>

GitHub Repository:

<https://github.com/Mohammad-Sami-codes/CSE443-Bioinformatics-Project->